Flutter Api Integration

1. Explain what a RESTful API is and its importance in mobile applications.

Ans : **Introduction**

In modern mobile application development, applications often need to communicate with servers to fetch, store, update, or delete data. One of the most commonly used methods for this communication is through a RESTful API. RESTful APIs enable mobile applications to interact efficiently with backend services and databases over the internet.

What is a RESTful API?

RESTful API stands for **Representational State Transfer Application Programming Interface**. It is an architectural style used for designing web services that allow different systems to communicate with each other using standard HTTP protocols.

A RESTful API works by allowing a client, such as a mobile application, to send requests to a server and receive responses in a structured format, usually JSON (JavaScript Object Notation). These APIs use standard HTTP methods such as:

- **GET** – Used to retrieve data from the server.
- **POST** – Used to send new data to the server.
- **PUT** – Used to update existing data.
- **DELETE** – Used to remove data from the server.

REST APIs are stateless, which means each request from the client contains all the information needed for the server to process it.

Features of RESTful APIs

RESTful APIs have several important characteristics:

1. **Stateless Communication** – Each request is independent and contains complete information.
2. **Client-Server Architecture** – The client and server operate separately, improving flexibility.
3. **Use of Standard HTTP Methods** – Communication is performed using well-defined HTTP operations.
4. **Data Format Support** – Mostly uses JSON, which is lightweight and easy to parse.
5. **Scalability** – Can handle large numbers of users and requests efficiently.

Importance of RESTful APIs in Mobile Applications

RESTful APIs play a critical role in mobile application development for several reasons:

1. Data Communication

Mobile applications need to exchange data with remote servers. RESTful APIs make this communication simple and efficient.

Example: A social media app fetching posts from a server.

2. Real-Time Data Access

REST APIs allow applications to access updated information from servers in real time.

Example: Weather apps displaying live weather updates.

3. Backend Integration

Mobile apps can connect with databases, authentication systems, and cloud services through REST APIs.

Example: Login systems connected to server-side authentication.

4. Platform Independence

REST APIs can be used by Android, iOS, and cross-platform applications like Flutter, making development easier.

5. Scalability and Flexibility

As the number of users grows, REST APIs can scale to handle increased traffic without major architectural changes.

6. Faster Development

Developers can use existing APIs instead of building backend functionality from scratch, reducing development time.

7. Secure Communication

REST APIs can implement authentication and authorization methods such as tokens and API keys to secure data transmission.

Example of REST API in Mobile Applications

Suppose an e-commerce mobile app needs to display products:

- The mobile app sends a **GET** request to the server.
- The server processes the request.
- The server returns product data in JSON format.
- The mobile app displays the products to the user.

Similarly, when a user places an order, the app sends a **POST** request to store order information on the server.

2. Describe how JSON data is parsed and used in Flutter.

Ans : **Introduction**

In mobile application development, data is often exchanged between applications and servers using JSON format. In Flutter, JSON data is commonly used when working with APIs, local files, or databases. Flutter provides powerful tools and libraries to parse JSON data and convert it into usable Dart objects.

What is JSON?

JSON stands for **JavaScript Object Notation**. It is a lightweight data-interchange format that is easy for humans to read and write, and easy for machines to parse and generate.

JSON stores data in **key-value pairs**.

Example of JSON data:

```
{
  "id": 1,
  "name": "Harshil",
  "email": "harshil@example.com"
}
```

Why JSON is Used in Flutter?

JSON is widely used in Flutter applications because:

1. It is lightweight and easy to transfer over the internet.
2. It is supported by most web APIs.
3. It is easy to convert into Dart objects.
4. It helps organize structured data efficiently.

How JSON Data is Parsed in Flutter

Parsing JSON in Flutter means converting JSON data into Dart objects so that the application can use the data.

The JSON parsing process generally involves the following steps:

Step 1: Fetch JSON Data

JSON data is usually received from a REST API using HTTP requests. Flutter commonly uses the http package package for this purpose.

Example workflow:

- Send a request to the server.
- Receive JSON response.
- Decode the response.

Step 2: Decode JSON Data

Flutter uses Dart's built-in **dart:convert** library to decode JSON.

The jsonDecode() method converts JSON string into Dart objects such as Map or List.

Example:

```
import 'dart:convert';

String jsonString = '{"name":"Harshil","age":22}';

Map<String, dynamic> data = jsonDecode(jsonString);

print(data["name"]);
```

In this example:

- JSON string is converted into a Dart Map.
- Values can be accessed using keys.

Step 3: Create Model Class

In Flutter, developers usually create model classes to represent JSON data in an organized way.

Example model:

```dart
class User {
  final int id;
  final String name;
  final String email;

  User({
    required this.id,
    required this.name,
    required this.email,
  });

  factory User.fromJson(Map<String, dynamic> json) {
    return User(
      id: json["id"],
      name: json["name"],
      email: json["email"],
    );
  }
}
```

This model class helps convert JSON data into Dart objects.

Step 4: Convert JSON into Dart Object

Using the fromJson() method, JSON data is converted into an object.

Example:

```dart
Map<String, dynamic> jsonData = {
  "id": 1,
  "name": "Harshil",
  "email": "harshil@example.com"
};

User user = User.fromJson(jsonData);

print(user.name);
```

Now the data can be accessed using object properties.

Step 5: Display Data in Flutter UI

After parsing, the data can be displayed in widgets such as Text, ListView, or Cards.

Example:

```
Text(user.name)
```

Parsing JSON Array

Sometimes APIs return multiple objects in a list.

Example JSON:

```
[
  {
    "id": 1,
    "name": "Harshil"
  },
  {
    "id": 2,
    "name": "Rahul"
  }
]
```

In Flutter, this can be parsed into a list of objects.

Example:

```
List<dynamic> jsonList = jsonDecode(response);

List<User> users =
    jsonList.map((item) => User.fromJson(item)).toList();
```

Use of Parsed JSON in Flutter Applications

Parsed JSON data is used in many types of Flutter applications, such as:

1. User Authentication

Used to receive user details after login.

2. E-commerce Applications

Used to display products, prices, and categories.

3. Social Media Applications

Used to load posts, comments, and user profiles.

4. Weather Applications

Used to show live weather information.

5. News Applications

Used to fetch and display news articles.

Advantages of JSON Parsing in Flutter

1. Easy integration with REST APIs.
2. Structured and organized data handling.
3. Improves code readability using model classes.
4. Supports complex nested data.
5. Makes UI updates easier.

3. Explain the purpose of HTTP methods (GET, POST, PUT, DELETE) and when to use each.

Ans : Introduction

In modern mobile and web application development, communication between client applications and servers is commonly performed using HTTP (HyperText Transfer Protocol). HTTP provides different request methods that define the type of operation to be performed on server data. The most commonly used HTTP methods in RESTful APIs are GET, POST, PUT, and DELETE. These methods help applications perform CRUD operations (Create, Read, Update, Delete) efficiently.

---

1. GET Method

Purpose

The **GET** method is used to retrieve data from a server. It requests information without modifying any existing data.

When to Use

GET should be used when the application needs to fetch or read data from a server.

Examples of Usage

- Fetching user profile information.
- Loading product lists in an e-commerce app.
- Retrieving weather updates.
- Displaying posts in a social media application.

Example

A mobile application requests product data from a server:

```
GET /products
```

The server responds with product information in JSON format.

Features of GET

- Used only for reading data.
- Does not change server data.
- Can be cached for faster performance.
- Parameters are usually sent in the URL.

---

2. POST Method

Purpose

The **POST** method is used to send new data to the server and create a new resource.

When to Use

POST should be used when the application needs to add or submit new data.

Examples of Usage

- User registration.
- User login requests.
- Creating a new order.
- Uploading comments or posts.

Example

A mobile application sends user registration data:

```
POST /users
```

Example request data:

```
{
  "name": "Harshil",
  "email": "harshil@example.com"
}
```

The server stores the new user information.

Features of POST

- Used for creating new resources.
- Sends data in the request body.
- Can modify server data.
- Usually not cached.

---

3. PUT Method

Purpose

The **PUT** method is used to update existing data on the server.

When to Use

PUT should be used when an application needs to replace or update an existing resource.

Examples of Usage

- Updating user profile information.
- Editing product details.
- Changing account settings.
- Updating delivery address.

Example

Updating user information:

```
PUT /users/1
```

Example request data:

```
{
  "name": "Harshil Shah",
  "email": "harshil@example.com"
}
```

The server updates the existing user record.

Features of PUT

- Used for updating existing resources.
- Sends updated data in the request body.

- Replaces existing resource data.
- Can be called multiple times with the same result.

---

4. DELETE Method

Purpose

The **DELETE** method is used to remove existing data from the server.

When to Use

DELETE should be used when an application needs to permanently remove a resource.

Examples of Usage

- Deleting user accounts.
- Removing products from a database.
- Deleting notes or messages.
- Cancelling stored records.

Example

Deleting a user:

```
DELETE /users/1
```

The server removes the user record.

Features of DELETE

- Used for removing resources.
- Permanently deletes server data.
- Usually requires authorization.
- Cannot always be reversed.

Importance in Flutter Applications

In Flutter applications, these HTTP methods are commonly used when working with APIs:

- **GET** → Fetch products, users, notes, posts.
- **POST** → Register users, add notes, create orders.
- **PUT** → Update profile or edit existing records.

- **DELETE** → Remove notes, posts, or user data.

Using these methods correctly helps applications communicate efficiently with backend servers.